

sAIfer Lab

Joint lab on Safety and Security of AI

Struttura iterativa for

La funzione range() e la sintassi del ciclo for

La funzione `range()`

- `range()` restituisce un oggetto iterabile che produce una sequenza di numeri interi
- Tre forme: `range(stop)`, `range(start, stop)`, `range(start, stop, step)`
- `start`: valore iniziale, incluso (default 0)
- `stop`: valore finale, escluso
- `step`: incremento tra due elementi (default 1)

`range` non è una lista

L'oggetto `range` memorizza solo inizio, fine (esclusa) e step.
Si converte in lista con `list()`.

```
print(list(range(5)))           # [0, 1, 2, 3, 4]
print(list(range(2, 6)))        # [2, 3, 4, 5]
print(list(range(0, 10, 3)))    # [0, 3, 6, 9]
print(list(range(4, -1, -1)))   # [4, 3, 2, 1, 0]
print(list(range(10, 0)))       # []
```

Step negativo e sequenze vuote

- Con `step` negativo la sequenza è decrescente: `start` deve essere maggiore di `stop`
- `stop` resta escluso: per arrivare a `0` serve `stop = -1`. `range(4, 0, -1)` produce solo `[4, 3, 2, 1]`
- Se `start` e `stop` non sono coerenti con il segno dello `step`, la sequenza è vuota
- Es. `range(10, 0)`: nessun errore, ma un `for` su di essa non esegue nessuna iterazione

Il ciclo `for`

- Statement di controllo del flusso che itera su una sequenza (iterabile), come una lista, una stringa o l'oggetto restituito da `range()`
- Diverso dal `for` di C o Java: non itera in base a un'espressione booleana, come fa il `while`
- Scorre direttamente gli **elementi** di un oggetto iterabile

Sintassi completa del `for`

```
for <variabile> in <iterabile>:
    <blocco_di_codice>
else:
    <blocco_opzionale_else>
```

Il blocco `else` è opzionale: viene eseguito una sola volta alla fine del ciclo, solo se il ciclo non è stato interrotto da un `break` (come per `while`)

Come funziona il for

A ogni iterazione la variabile prende l'**elemento** (non l'indice) successivo; il ciclo finisce quando gli elementi sono finiti

```
frutti = ['mela', 'pera', 'banana']

for frutto in frutti:
    print(frutto)
```

Output

```
mela
pera
banana
```

Come funziona il `for`

Con il `while` serve un indice gestito a mano:

```
frutti = ['mela', 'pera', 'banana']

i = 0
while i < len(frutti):
    print(frutti[i])
    i = i + 1
```

Output

```
mela
pera
banana
```

Il for su una stringa

Gli elementi di una stringa sono i singoli caratteri: una stringa viene scorsa carattere per carattere

```
for c in 'ciao':
    print(c)
```

Output

```
c
i
a
o
```

Scorrere gli elementi o gli indici

- `for el in lista`: scorre gli elementi. È la forma più semplice, quando serve solo il valore
- `for i in range(len(lista))`: scorre gli indici, da `0` a `len(lista) - 1`
- Gli indici servono quando conta la posizione: per modificare la lista o accedere all'elemento precedente o successivo

Elementi e indici: un esempio

```
voti = [24, 18, 30]

# scorro gli elementi
for voto in voti:
    print(voto)

# scorro gli indici
for i in range(len(voti)):
    print('esame', i, 'voto', voti[i])
```

Output

```
24
18
30
esame 0 voto 24
esame 1 voto 18
esame 2 voto 30
```

Scorrere gli indici al contrario

Il passo negativo serve per scorrere gli indici di una lista dall'ultimo al primo

```
lettere = ['a', 'b', 'c', 'd']

for i in range(len(lettere) - 1, -1, -1):
    print(i, lettere[i])      # 3 d, 2 c, 1 b, 0 a
```

Documentazione:
docs.python.org/3/library/stdtypes.html#range

La variabile del ciclo non modifica la lista

Assegnare un nuovo valore alla variabile del ciclo non modifica la lista: per modificarla bisogna passare dagli indici

```
voti = [24, 18, 30]

for voto in voti:
    voto = voto + 1          # cambia solo la variabile voto
print(voti)                  # [24, 18, 30]

for i in range(len(voti)):
    voti[i] = voti[i] + 1    # modifica l'elemento della lista
print(voti)                  # [25, 19, 31]
```



sAIfer Lab

Joint lab on Safety and Security of AI

Controllare e usare il ciclo for

break ed else; elemento precedente e successivo

break nel for

break

- Come nel caso di `while`, interrompe immediatamente il ciclo e passa all'istruzione dopo il `for`
- Nei cicli annidati interrompe solo il ciclo più interno in cui si trova

break: uscire dal ciclo

```
numeri = [4, 7, -2, 9, 5]

# break: stampo i numeri fino al primo negativo,
# poi esco dal ciclo
for el in numeri:
    if el < 0:
        print('trovato un negativo:', el)
        break
    print(el)
```

Stampa `4`, `7` e `trovato un negativo: -2`
I numeri `9` e `5` non vengono esaminati

`break` con `else`: esito di una ricerca

Il `break` si combina spesso con l'`else` del ciclo per sapere se una ricerca è andata a buon fine

```
numeri = [4, 7, 2, 9, 5]

for el in numeri:
    if el < 0:
        print('trovato un negativo:', el)
        break
else:
    # eseguito solo se il ciclo non è stato interrotto
    print('nessun numero negativo')
```

Elemento precedente o successivo

- Scorrendo una lista può servire, oltre all'elemento corrente, quello precedente o successivo
- Il primo elemento (indice `0`) non ha un precedente
- L'ultimo (indice `len(lista) - 1`) non ha un successivo
- Per evitare un `IndexError`: limitare gli indici del ciclo, oppure gestire questi casi in modo esplicito

Precedente e successivo con il for

```
numeri = [10, 20, 30, 40]

# for (elemento successivo)
for i in range(len(numeri) - 1): # evito l'ultimo
    corrente = numeri[i]
    successivo = numeri[i + 1]
    print(corrente, successivo)
```

Output

```
10 20
20 30
30 40
```

Precedente e successivo con il for

```
numeri = [10, 20, 30, 40]

# for (elemento precedente)
for i in range(1, len(numeri)): # evito il primo, parto da 1
    precedente = numeri[i - 1]
    corrente = numeri[i]
    print(precedente, corrente)
```

Output

```
10 20
20 30
30 40
```

Precedente e successivo con il `while`

```
numeri = [10, 20, 30, 40]

# while (elemento successivo)
i = 0
while i < len(numeri) - 1:
    corrente = numeri[i]
    successivo = numeri[i + 1]
    print(corrente, successivo)
    i = i + 1

# while (elemento precedente)
i = 1
while i < len(numeri):
    precedente = numeri[i - 1]
    corrente = numeri[i]
    print(precedente, corrente)
    i = i + 1
```

Esempio: verifica dell'ordinamento

Per verificare se un sequenza è ordinata in senso crescente o decrescente si possono confrontare tutte le coppie di elementi adiacenti (primo e secondo, secondo e terzo, ecc.)

```
numeri = [4, 7, 2, 9, 5]
ordinati = True  # assumo che siano ordinati

# verifica dell'ordinamento in senso crescente:
# se un qualsiasi elemento è maggiore del successivo,
# non sono ordinati (e posso interrompere il ciclo)
for i in range(len(numeri) - 1):  # fino al penultimo
    if numeri[i] > numeri[i + 1]:
        ordinati = False
        break
if ordinati:
    print('Sono ordinati in senso crescente')
else:
    print('Non sono ordinati in senso crescente')
```

Esempio: verifica dell'ordinamento (con `else`)

Stesso procedimento della versione precedente, ma l'uso di `else` con `for` consente di evitare la variabile `ordinati`

```
numeri = [4, 7, 2, 9, 5]

for i in range(len(numeri) - 1):  # fino al penultimo
    if numeri[i] > numeri[i + 1]:
        print('Non sono ordinati in senso crescente')
        break
else:
    # eseguito solo se il ciclo non è stato interrotto
    print('Sono ordinati in senso crescente')
```



sAIfer Lab

Joint lab on Safety and Security of AI

Esercizi

Dove possibile, una soluzione con for e una con while

Indicazioni per gli esercizi

- Dove possibile, scrivete due soluzioni: una con `for` e una con `while`
- Gestite le uscite anticipate con `if` concatenati o operatori booleani
- Dove opportune, usate anche `break` e la clausola `else`
- Alcuni esercizi sono gli stessi del `while`: notate come alcune soluzioni diventano più semplici con il `for`

Stampa al contrario (`for`)

Stampare una lista in ordine inverso usando `for`

```
numeri = [1, 2, 3, 4, 5]
n = len(numeri)
for i in range(n - 1, -1, -1):
    print(numeri[i])
```

Conta le occorrenze di una parola (for)

Data una lista di stringhe, chiedere una parola da cercare e stampare quante volte è presente nella lista

```
parole = ['cane', 'gatto', 'gatto', 'topo', 'gatto']

cerca = input("Quale parola vuoi contare? ")
conteggio = 0

for el in parole:
    if el == cerca:
        conteggio = conteggio + 1

print(f"La parola '{cerca}' compare "
      f"{conteggio} volte nella lista.")
```

Lista senza duplicati (for)

Data una lista che può contenere elementi duplicati, creare una nuova lista con gli stessi elementi, ma senza duplicati

```
numeri = [1, 2, 3, 2, 3, 4, 3, 4, 5, 6, 7]

unici = []

for el in numeri:
    if el not in unici:
        unici = unici + [el]

print("Lista senza duplicati:", unici)
```

Media degli elementi in posizione pari (for)

Calcolare la media aritmetica dei numeri in posizione pari (indice 0, 2, 4…)

```
numeri = [1, 4, 3 , 8, 5, 3, 7, 1, 9] # media = 5

somma = 0
conta = 0

for i in range(0, len(numeri), 2):
    somma = somma + numeri[i]
    conta = conta + 1

media = somma/conta

print("Media dei numeri in posizione pari:", media)
```

Parola palindroma (for)

Verificare se una parola (stringa) è palindroma (uguale da sinistra e da destra)

```
parola = 'ingegni'
# parola = 'ingegno'

for i in range(int(len(parola)/2)):
    j = len(parola) – i - 1
    if parola[i] != parola[j]:
        print("La parola NON è palindroma.")
        break
else:
    print("La parola è palindroma!")
```

Conta pari e dispari

Data una lista, contare il numero di elementi pari ed elementi dispari

```
lista = [1, 2, 3, 4, 5, 6, 7, 8, 9]

n_pari = 0
n_dispari = 0

for el in lista:
    if el % 2 == 0:
        n_pari = n_pari + 1
    else:
        n_dispari n_dispari + 1

print('pari: ', n_pari, 'dispari: ', n_dispari )
```

Elementi maggiori di 10

Data una lista, stampare gli elementi maggiori di 10

```
lista = [1, 3, 5, 10, 21, 44]

for el in lista:
    if el > 10:
        print(el)
```

Raddoppia i negativi in una nuova lista

Data una lista, creare una nuova lista raddoppiando i valori negativi e lasciando inalterati i valori positivi

```
lista = [1, -3, 5, 10, -4, 4]

lista2 = []

for el in lista:
    if el > 0:
        lista2 = lista2 + [el]
    else:
        lista2 = lista2 + [2*el]

print(lista2)
```

Raddoppia i negativi nella stessa lista

Data una lista, raddoppiare i valori negativi e lasciare inalterati i valori positivi

```
lista = [1, -3, 5, 10, -4, 4]

for i in range(len(lista)):
    if lista[i] < 0:
        lista[i] = lista[i]*2

print(lista)
```

Posizione del primo numero negativo

Trovare la posizione del primo numero negativo in una lista

```
lista = [1, -3, 5, 10, -4, 4]
# lista = [1, 2, 3, 4, 5]

trovato = False
for i in range(len(lista)):
    if lista[i] < 0:
        i_neg = i
        trovato = True
        break
if trovato:
    print('posizione del primo numero negativo:', i_neg)
else:
    print('nessun valore negativo')
```

Conta le vocali

Contare le vocali in una lista di lettere

```
lettere = ['l', 'e', 'o', 'p', 'a', 'r', 'd', 'o']
vocali = ['a', 'e', 'i', 'o', 'u']

conteggio = 0

for l in lettere:
    if l in vocali:
        conteggio = conteggio + 1

print("Numero di vocali:", conteggio)
```

Elementi comuni a due liste

Confrontare due liste e trovare gli elementi comuni.
Il programma deve produrre una lista contenente gli elementi comuni, SENZA duplicati

```
lista1 = [7, 11, 13, 19]
lista2 = [2, 3, 5, 7, 19, 23, 29]

comuni = []

for el in lista1:
    if el in lista2 and el not in comuni:
        comuni = comuni + [el]

print("Elementi comuni:", comuni)
```

Inserimento di numeri fino a 0 (`while`)

Chiedere all'utente di inserire numeri finché digita 0. Salvare i numeri in una lista e stamparla alla fine.

```
numeri = []
print("Inserisci un numero (0 per terminare): ")
numero = int(input())

while numero != 0:
    numeri = numeri + [numero]
    numero = int(input(
        "Inserisci un altro numero (0 per terminare): "))

print("Hai inserito:", numeri)
```

Somma dei numeri pari (`while`)

Inserire numeri finché l'utente scrive -1. Salvare i numeri pari in una lista e calcolare la loro somma.

```
pari = []
numero = int(input(
            "Inserisci un numero (-1 per terminare): "))

while numero != -1:
    if numero % 2 == 0:
        pari = pari + [numero]
    numero = int(input(
                "Inserisci un numero (-1 per terminare): "))

print("Numeri pari:", pari)
```

(cont.)

Somma dei numeri pari (`while`): la somma

...poi si sommano gli elementi della lista `pari`:

```
somma = 0
i = 0
while i < len(pari):
    somma = somma + pari[i]
    i = i + 1

print("Somma dei pari:", somma)
```

Lista al contrario (`while`)

Chiedere un sequenza di numeri di una data lunghezza all'utente, poi stampare la lista in ordine inverso con `while`

```
numeri = []
i = 0

# imposto un valore fisso di n
# per non avere la scomodità dell'input
n = 5
while i < n:
    num = int(input(f"Inserisci il numero {i+1}: "))
    numeri = numeri + [num]
    i = i + 1
```

(cont.)

Lista al contrario (`while`): la stampa

```
# Stampa al contrario con while
i = len(numeri) - 1
print("Numeri al contrario:")
while i >= 0:
    print(numeri[i])
    i = i - 1

# soluzione alternativa
i = -1
print("Numeri al contrario:")
while i >= -n:
    print(numeri[i])
    i = i - 1
```

Conta le occorrenze di una parola (`while`)

Data una lista di stringhe, chiedere una parola da cercare e stampare quante volte è presente nella lista

```
parole = ['cane', 'gatto', 'gatto', 'topo', 'gatto']

cerca = input("Quale parola vuoi contare? ")
conteggio = 0
i = 0

while i < len(parole):
    if parole[i] == cerca:
        conteggio = conteggio + 1
    i = i + 1

print(f"La parola '{cerca}' compare "
      f"{conteggio} volte nella lista.")
```

Lista senza duplicati (`while`)

Acquisire una sequenza di numeri finché l'utente digita -1 e memorizzarla in una lista. Creare poi una nuova lista contenente gli stessi valori, senza duplicati.

```
numeri = []
numero = int(input(
    "Inserisci un numero (-1 per terminare): "))

while numero != -1:
    numeri.append(numero)
    numero = int(input(
        "Inserisci un numero (-1 per terminare): "))
```

(cont.)

Lista senza duplicati (`while`): i valori unici

```
unici = []
i = 0
while i < len(numeri):
    if numeri[i] not in unici:
        unici = unici + [numeri[i]]
    i = i + 1

print("Lista senza duplicati:", unici)
```

Media degli elementi in posizione pari (`while`)

Chiedere 10 numeri. Calcolare la media aritmetica dei numeri in posizione pari (indice 0, 2, 4…).

```
numeri = []
i = 0

while i < 10:
    num = int(input("Inserisci il numero " + str(i+1) + ":"))
    numeri = numeri + [num]
    i = i + 1
```

(cont.)

Media in posizione pari (`while`): il calcolo

```
somma = 0
conta = 0
i = 0

while i < len(numeri):
    if i % 2 == 0:
        somma = somma + numeri[i]
        conta = conta + 1
    i = i + 1

media = somma/conta

print("Media dei numeri in posizione pari:", media)
```

Parola palindroma (`while`)

Verifica se una stringa forma una parola palindroma

```
parola = 'ingegni'
# parola = 'ingegno'
i = 0  # indice che parte da sinistra
j = len(parola) - 1  # indice che parte da destra
palindroma = True
while i < j:
    if parola[i] != parola[j]:
        palindroma = False
        break
    i = i + 1
    j = j - 1
if palindroma:
    print("La parola è palindroma!")
else:
    print("La parola NON è palindroma.")
```